

Disciplina: Qualidade de Software e Testes

Professor: Leonardo Cardia

Plano de Contingência e Refatoração

Gilded Rose — Modernização de Sistema Legado

Discentes:

Andre Junio Deomondes Carvalhal

Gianluca Motta Lopo

Igor Lima Carvalho

Lucca Torres Badaró Silvani

Ryan Luigi Paixão da Luz

2026

Plano de Contingência e Refatoração

Estratégia geral

Refatoração incremental com rede de testes de caracterização antes de qualquer mudança estrutural. Nenhum passo de refatoração acontece sem testes verdes cobrindo o comportamento atual primeiro (Passo 6 antes do Passo 7, [[docs/requisitos.md]]).

O que será alterado

- `update_quality` será decomposto: lógica de cada tipo de item movida para uma estratégia isolada (camada domain), seguindo Clean Architecture.
- `GildedRose` passa a delegar a uma função/mapa de resolução de estratégia por nome de item, em vez de condicionais inline.
- Será adicionada a estratégia para itens `Conjured` (ausente hoje — ver docs/`diagnostico_tecnico.md` item 7).
- Será adicionada suíte de testes unitários por tipo de item, mais configuração correta do reporter de approval testing (hoje falha por falta de configuração, não por bug de lógica).

O que NÃO será alterado (restrição obrigatória)

- Classe `Item` (`legacy/gilded_rose.py:39-46`) — assinatura, atributos e `__repr__` permanecem idênticos.
- Propriedade `items` da classe `GildedRose` — continua sendo uma lista simples de `Item`, atribuída no `__init__`, sem encapsulamento adicional.
- Comportamento observável para itens comuns, `Aged Brie`, `Sulfuras` e `Backstage Passes` — validado contra `docs/evidencias/execucao_legacy_antes.txt`.

Riscos identificados e ações preventivas

Risco	Ação preventiva
Quebrar regra de negócio existente ao mover lógica pra estratégias separadas	Testes de caracterização escritos e passando ANTES de tocar no código (Passo 6); rodar fixture 30 dias antes/depois e comparar diff
Alterar <code>Item</code> ou <code>items</code> por engano (penalidade -2,0 no barema)	Revisão de cada commit de refatoração confirma que <code>legacy/gilded_rose.py</code> (classe <code>Item</code>) não é tocado; testes incluem verificação de que <code>GildedRose(items).items is items</code>
Implementar <code>Conjured</code> incorretamente (penalidade -2,0)	Testes dedicados cobrindo: degradação 2/dia antes do vencimento, 4/dia depois, nunca abaixo de 0 — RF11 em docs/ <code>requisitos.md</code>
	Testes de caracterização cobrem limites antes da refatoração; novos testes mantêm os mesmos casos

Risco	Ação preventiva
Perder cobertura de caso de borda (quality em 0 ou 50, Backstage no dia do show)	
Refatoração grande demais, dificultando revisão e rollback	Commits pequenos, um passo por commit (extrair função → introduzir estratégia → remover condicional antigo), cada commit com testes passando

Ordem de execução

1. Testes de caracterização cobrindo todos os tipos de item existentes, rodando contra o código legado intocado (branch test/caracterizacao).
2. Extrair lógica de update_quality para camada domain, mantendo testes verdes a cada passo (branch refactor/clean-architecture).
3. Implementar estratégia Conjured e seus testes (branch feat/conjured).
4. Rodar suíte completa, gerar evidência pós-refatoração, comparar com evidência pré-refatoração (branch test/cobertura-final).

Critérios de validação

- Toda a suíte de testes (caracterização + novos testes unitários) deve passar antes de qualquer merge em main.
- Saída do fixture de 30 dias deve ser idêntica à evidência pré-refatoração para todos os itens exceto Conjured (que estava com bug).
- `git diff` da classe Item entre a versão legada e a versão final deve ser vazio.